

Introduction to Statistical Methods for Data Science

Modelling Cloud Server Power Consumption using Polynomial Regression

Submitted by

Rohan Pokhrel | 260537@softwarica.edu.np

Github link: `https://github.com/Rotavirus22/STW7089CEM_Coursework`

Contents

Abstract **1**

1 Introduction **2**

2 Data and Methodology **2**

2.1 Data preparation 2

2.2 Candidate models 3

2.3 Statistical methods 3

3 Task 1: Preliminary Data Analysis **4**

3.1 Task 1.1: Time series plots 4

3.2 Task 1.2: Distribution of each signal 6

3.3 Task 1.3: Correlation and scatter plots 7

3.4 Baseline linear model 9

4 Task 2: Regression – Modelling the Relationship between Server Metrics **10**

4.1 Task 2.1: Least squares parameter estimates 10

4.2 Task 2.2: Residual sum of squares 10

4.3 Task 2.3: Log-likelihood 10

4.4 Task 2.4: AIC and BIC 11

4.5 Task 2.5: Distribution of the residuals 11

4.6 Task 2.6: Selection of the best model 13

4.7 Task 2.7: Train/test validation and confidence intervals 13

5 Task 3: Approximate Bayesian Computation (ABC) **15**

5.1 Task 3.1: Choice of parameters 16

5.2 Task 3.2: Uniform priors 16

5.3 Task 3.3: Rejection ABC 16

5.4 Task 3.4: Joint and marginal posterior distributions 17

5.5 Task 3.5: Explanation of the results 17

6 Discussion **18**

7 Conclusion **19**

References **21**

A R Code **22**

A.1 Task 1: Preliminary data analysis 22

A.2 Task 2: Regression (Tasks 2.1–2.7) 25

A.3 Task 3: Approximate Bayesian Computation 30

List of Figures

1 Time series of the four inputs and the output power over the full 1000 s record. . 5

2 Zoomed time series of the first 20 s (200 samples). 6

3 Histograms with kernel density estimates (red) for each signal. 7

4 Pearson correlation matrix of all signals. 8

5 Scatter plots of the output power against each input, with LOWESS curves (red). 9

6 Residuals vs fitted values and normal Q-Q plot for the baseline linear model. . . 9

7 Residual histograms (left) and normal Q-Q plots (right) for the five candidate models. 12

8 Predicted vs actual power for all 3000 test samples (Model 2 trained on 70% of the data). 14

9 Model 2 predictions with 95% confidence intervals for the first 40 test samples. Top: actual vs predicted power. Bottom: zoomed view of actual minus predicted, with 95% confidence-interval error bars (blue) and 95% prediction interval (dashed green). 15

10 Rejection ABC results for Model 2: joint posterior of $(\theta_{bias}, \theta_{x_4})$ (left) and marginal posteriors (centre, right). The red markers show the least squares estimates. . . . 17

List of Tables

1 Summary statistics of the five signals ($n = 10{,}000$). 4

2 Correlation of each input with the output y. 7

3 Least squares estimates $\hat{\boldsymbol{\theta}}$ for the five candidate models (all data). 10

4 Model comparison criteria ($n = 10{,}000$). The lowest AIC and BIC (selected model) are in bold. 11

5 Posterior summaries from 1921 accepted ABC samples. 17

Abstract

This report looks for the polynomial regression model that best explains the total power consumption of a simulated cloud server from four telemetry signals: CPU utilisation (x_1), memory usage (x_2), disk I/O (x_3) and network traffic (x_4). The dataset has 10,000 samples taken every 0.1 s. Exploratory analysis showed that power is strongly and non-linearly related to disk I/O, weakly related to CPU utilisation, and almost unrelated to memory usage and network traffic when each is looked at on its own. Five candidate models were fitted by least squares and compared using the residual sum of squares, the Gaussian log-likelihood, AIC, BIC and the shape of the residual distribution. Model 2, $y = \theta_1 x_4 + \theta_2 x_1^3 + \theta_3 x_3^4 + \theta_{bias}$, had the lowest AIC and BIC and normally distributed residuals. On a 30% hold-out test set it reached an RMSE of 5.04 W and $R^2 = 0.99996$. Rejection ABC on its two largest parameters gave narrow posteriors centred on the least squares values, with a clear negative correlation between the intercept and the network coefficient.

1 Introduction

Data centres use a large and growing share of the world's electricity, and most of this energy is drawn by the servers themselves. Knowing how much power a server draws under a given workload helps operators plan power provisioning, place virtual machines on hosts, and spot hardware that behaves abnormally (Fan et al., 2007). Power can be measured directly with hardware meters, but these are expensive and hard to deploy on every machine. A cheaper option is to predict power from software telemetry, such as CPU, memory, disk and network load, which every operating system already reports. This only works if the mathematical link between the load signals and the power draw is known. Real servers are also not *energy proportional*: they use a large share of their peak power even when idle, and power grows non-linearly with load (Barroso and Hölzle, 2007). A simple straight-line model is therefore unlikely to be enough.

The coursework provides simulated telemetry for one server in three files:

- `X.csv`: four input signals, x_1 (CPU utilisation), x_2 (memory usage), x_3 (disk I/O) and x_4 (network traffic);
- `y.csv`: the output signal y, the total power consumption;
- `time.csv`: the sampling time of each row, in seconds.

All five signals contain additive, independent, zero-mean Gaussian noise of unknown variance.

The aim is to find which of five given polynomial models "truly" describes the relationship between the inputs and y. The report follows the three coursework tasks:

1. **Task 1** – exploratory analysis of the signals: time series, distributions, correlations and scatter plots (Section 3).
2. **Task 2** – least squares fitting of the five candidate models, model comparison with RSS, log-likelihood, AIC, BIC and residual analysis, and validation of the chosen model on a train/test split (Section 4).
3. **Task 3** – rejection Approximate Bayesian Computation (ABC) to estimate the posterior distribution of the two largest parameters of the chosen model (Section 5).

Section 6 discusses the results and their limits, and Section 7 concludes. All analysis was done in R version 4.6.1 (R Core Team, 2026) using only base R functions. The full annotated code is in Appendix A, and the code that produced each result is referenced in the text.

2 Data and Methodology

2.1 Data preparation

The three CSV files were read into R and joined column by column into one data frame with the columns `time`, `x1`, `x2`, `x3`, `x4` and `y` (Appendix A.1). There are $n = 10{,}000$ rows, sampled

every 0.1 s from 0 to 999.9 s. No value was missing, so no rows were removed or imputed.

2.2 Candidate models

The five candidate models given in the brief are:

$$\begin{aligned}
&\text{Model 1:} \quad y = \theta_1 x_4 + \theta_2 x_1^2 + \theta_3 x_1^3 + \theta_4 x_2^4 + \theta_5 x_1^4 + \theta_{bias} + \varepsilon \\
&\text{Model 2:} \quad y = \theta_1 x_4 + \theta_2 x_1^3 + \theta_3 x_3^4 + \theta_{bias} + \varepsilon \\
&\text{Model 3:} \quad y = \theta_1 x_3^3 + \theta_2 x_3^4 + \theta_{bias} + \varepsilon \\
&\text{Model 4:} \quad y = \theta_1 x_2 + \theta_2 x_1^3 + \theta_4 x_3^4 + \theta_{bias} + \varepsilon \\
&\text{Model 5:} \quad y = \theta_1 x_4 + \theta_2 x_1^2 + \theta_3 x_1^3 + \theta_4 x_3^4 + \theta_{bias} + \varepsilon
\end{aligned}$$

The brief numbers the parameters of Model 4 as $\theta_1, \theta_2, \theta_4$ with no θ_3. This looks like a typing error, so Model 4 is treated as having three regressors plus a bias, that is $k = 4$ parameters.

Each model is non-linear in the inputs but *linear in the parameters*. Each can therefore be written as $\mathbf{y} = \mathbf{X}\boldsymbol{\theta} + \boldsymbol{\varepsilon}$, where each column of the design matrix $\mathbf{X}$ is one transformed input (for example x_1^3) and the last column is all ones for the bias.

2.3 Statistical methods

Least squares. The parameters are estimated by ordinary least squares,

$$\hat{\boldsymbol{\theta}} = (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{y}, \tag{1}$$

which is also the maximum likelihood estimate when the noise is i.i.d. Gaussian (Montgomery et al., 2012).

Goodness of fit. The residual sum of squares is $\mathrm{RSS} = \sum_{i=1}^{n}(y_i - \mathbf{x}_i\hat{\boldsymbol{\theta}})^2$. With the noise variance estimated as $\hat{\sigma}^2 = \mathrm{RSS}/(n-1)$, as the brief specifies, the Gaussian log-likelihood is

$$\ln p(D \mid \hat{\boldsymbol{\theta}}) = -\frac{n}{2}\ln(2\pi) - \frac{n}{2}\ln(\hat{\sigma}^2) - \frac{1}{2\hat{\sigma}^2}\mathrm{RSS}. \tag{2}$$

Information criteria. Adding parameters always lowers the RSS, so models of different sizes are compared with

$$\mathrm{AIC} = 2k - 2\ln p(D \mid \hat{\boldsymbol{\theta}}), \qquad \mathrm{BIC} = k\ln(n) - 2\ln p(D \mid \hat{\boldsymbol{\theta}}), \tag{3}$$

where k is the number of estimated parameters (Akaike, 1974; Schwarz, 1978). Lower values are better. BIC penalises each extra parameter by $\ln(10000) \approx 9.2$ instead of 2, so it favours smaller models more strongly than AIC.

Residual checks. If a model has the right structure, its residuals should look like the Gaussian noise in the data. This was checked with histograms, normal Q-Q plots and the Shapiro–Wilk test (Shapiro and Wilk, 1965).

Validation and intervals. The chosen model was refitted on a random 70% of the data and tested on the other 30%. For a test input $\mathbf{x}_*$, the 95% confidence interval of the mean prediction is $\hat{y}_* \pm t_{0.975,\, n_{tr}-k}\sqrt{\mathbf{x}_*^T \hat{\boldsymbol{\Sigma}}_\theta \mathbf{x}_*}$, with $\hat{\boldsymbol{\Sigma}}_\theta = \hat{\sigma}^2(\mathbf{X}^T\mathbf{X})^{-1}$. The 95% prediction interval for a new observation adds the noise variance $\hat{\sigma}^2$ inside the square root (James et al., 2013).

Rejection ABC. Parameter values are drawn from a prior. Each draw is used to generate model output, and a draw is kept only if that output is close enough to the observed data. The kept draws approximate the posterior distribution (Pritchard et al., 1999; Beaumont et al., 2002; Sunnåker et al., 2013). Section 5 gives the details.

3 Task 1: Preliminary Data Analysis

Before fitting any model, the five signals were explored to understand their range, shape and dependencies. Table 1 gives the summary statistics, produced by the `summary()`, `sd()` and skewness lines of the Task 1 script (Appendix A.1).

Table 1: Summary statistics of the five signals ($n = 10{,}000$).

Signal	Min	Q1	Median	Mean	Q3	Max	Skewness
x_1 CPU	1.00	2.72	4.45	4.46	6.18	8.00	0.03
x_2 Memory	2.00	3.78	5.54	5.53	7.30	9.00	−0.02
x_3 Disk I/O	−0.78	3.99	5.01	5.00	6.01	11.72	0.01
x_4 Network	0.00	0.87	2.08	3.00	4.14	27.36	1.94
y Power	121.7	507.1	828.3	1084.1	1375.9	15241.3	2.71

3.1 Task 1.1: Time series plots

Figure 1 plots every signal against time over the full 1000 s. With 10,000 points the plots turn into dense bands. They still show that no signal has a trend, a level shift or a change in spread over time: all five are stationary. Disk I/O (x_3) varies around a level of about 5. Network traffic (x_4) sits near zero with frequent short spikes. Power (y) shows the same spiky shape, with a few very large peaks of up to about 15,000 W.

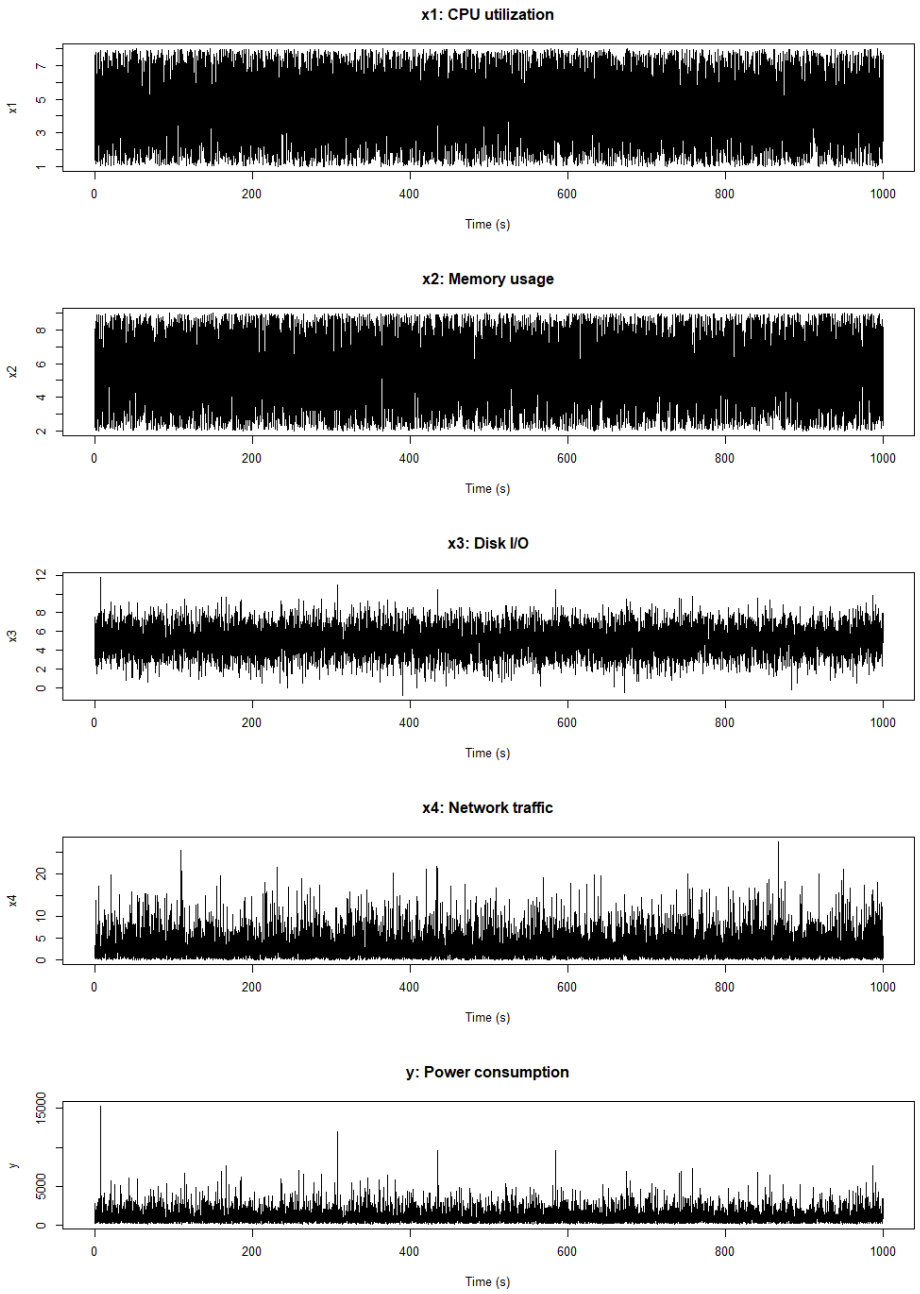


Figure 1: Time series of the four inputs and the output power over the full 1000 s record.

Figure 2 zooms in on the first 20 s (200 samples) so that single samples can be seen. Neighbouring samples are not similar: each value looks like a fresh random draw, with no cycles or smooth movements. The lag-1 autocorrelation of every signal is between -0.012 and 0.011, which confirms this. The samples can therefore be treated as independent observations, which the regression assumption of i.i.d. errors needs. The time stamp carries no useful information for modelling. The plot also shows that the large power spike at about 7 s happens at the same time as a spike in disk I/O, which is a first hint that y depends strongly on x_3.

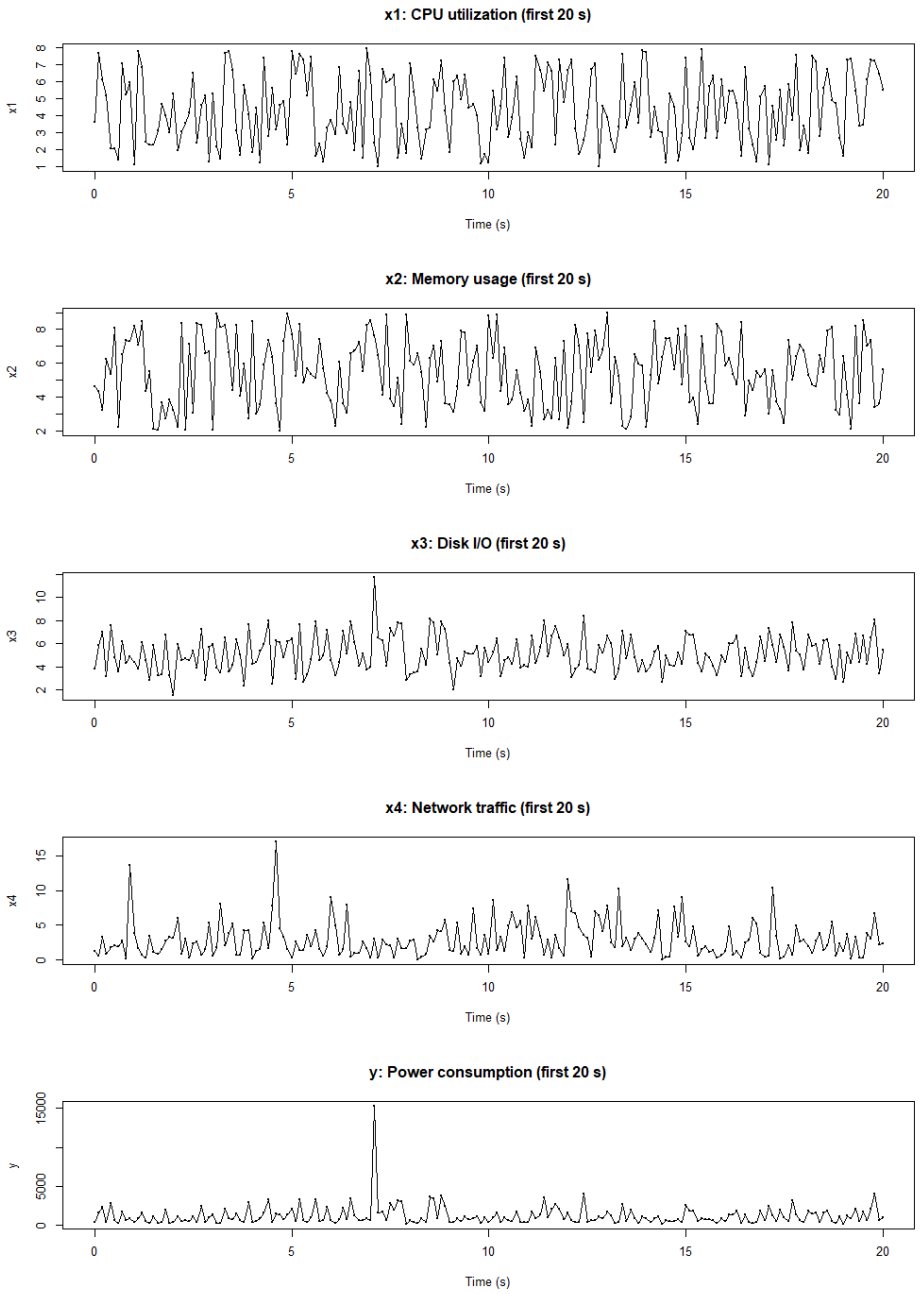


Figure 2: Zoomed time series of the first 20 s (200 samples).

3.2 Task 1.2: Distribution of each signal

Figure 3 shows a histogram of each signal with a kernel density curve. The four inputs have clearly different shapes:

- x_1 (CPU) and x_2 (memory) are flat between about 1–8 and 2–9. They look **uniformly distributed**, and the dips at the edges of the density curve are only a smoothing effect.
- x_3 (disk I/O) is bell-shaped and symmetric around 5 (skewness 0.01). It looks **normally distributed**, with standard deviation about 1.5.

- x_4 (network) is strongly **right-skewed** (skewness 1.94). Its mean (3.0) equals its standard deviation (3.0) and it never goes below zero, which matches an exponential distribution.
- y (power) is also strongly right-skewed (skewness 2.71) with a long upper tail.

The output is skewed even though x_3 is symmetric, which suggests that y depends on a high power of an input: raising a symmetric variable to the fourth power stretches its upper tail.

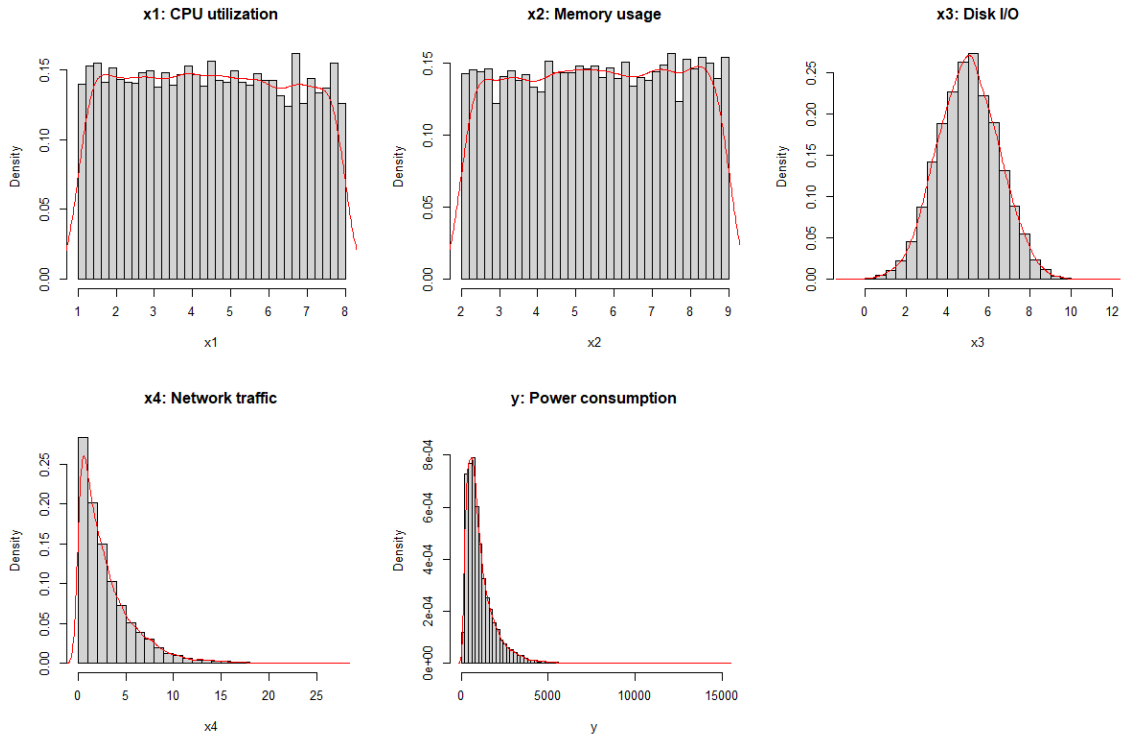


Figure 3: Histograms with kernel density estimates (red) for each signal.

3.3 Task 1.3: Correlation and scatter plots

Table 2 and Figure 4 give the correlation between all pairs of signals. The inputs are almost uncorrelated with each other ($|r| \leq 0.023$), so there is no multicollinearity and the effect of each input can be estimated separately. Against the output:

- x_3 has a strong positive correlation ($r = 0.851$). Its Spearman rank correlation is even higher (0.931), which shows that the relationship is monotonic but *not* linear.
- x_1 has a weak positive correlation ($r = 0.174$, Spearman 0.291).
- x_2 ($r = -0.014$) and x_4 ($r = 0.010$) show no visible correlation.

Table 2: Correlation of each input with the output y.

	x_1	x_2	x_3	x_4
Pearson r with y	0.174	−0.014	0.851	0.010
Spearman ρ with y	0.291	−0.012	0.931	0.016

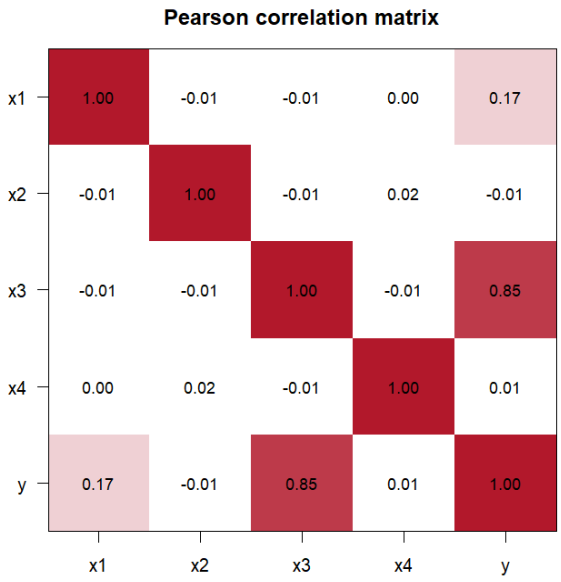


Figure 4: Pearson correlation matrix of all signals.

The scatter plots in Figure 5 show the shape of each relationship. The plot of y against x_3 shows a clear, tight, upward curve that gets steeper as x_3 grows, which is typical of a high-order polynomial term. The plot against x_1 shows a lower boundary of the point cloud that rises with x_1, so x_1 has a smaller non-linear effect that is hidden by the spread caused by x_3. The plots against x_2 and x_4 have flat LOWESS lines.

A low correlation does not prove that an input has no effect. The spread in y caused by x_3 (standard deviation 884 W) is so large that a small linear effect of x_4 cannot be seen in a two-variable plot. This is why the models must be compared with all inputs together, as in Task 2.

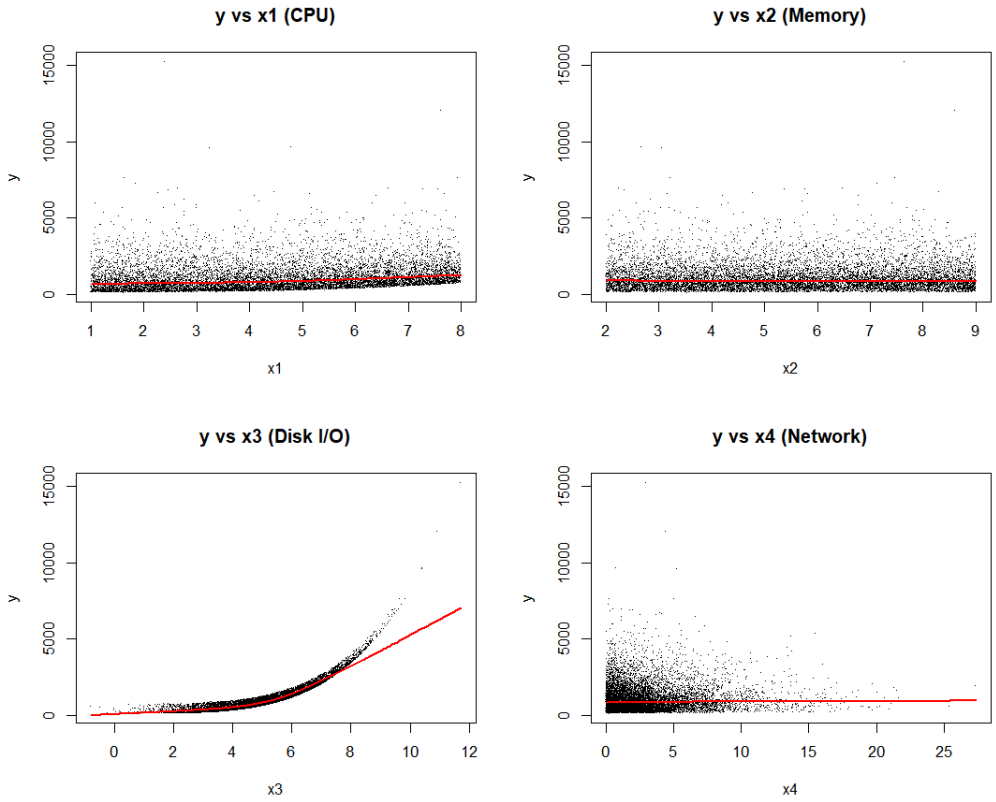


Figure 5: Scatter plots of the output power against each input, with LOWESS curves (red).

3.4 Baseline linear model

As a reference, a linear model with all four inputs was fitted with `lm()`. It explains 75.9% of the variance ($R^2 = 0.7587$) with a residual standard error of 434.1 W. x_1, x_3 and x_4 are significant ($p < 0.001$), but x_2 is not ($p = 0.46$). The residual plots in Figure 6 show that the linear model is not adequate. The residuals form a curved pattern against the fitted values, and the Q-Q plot bends sharply upwards in the right tail. Both signs mean that the true relationship is non-linear, which supports using the polynomial models of Task 2.

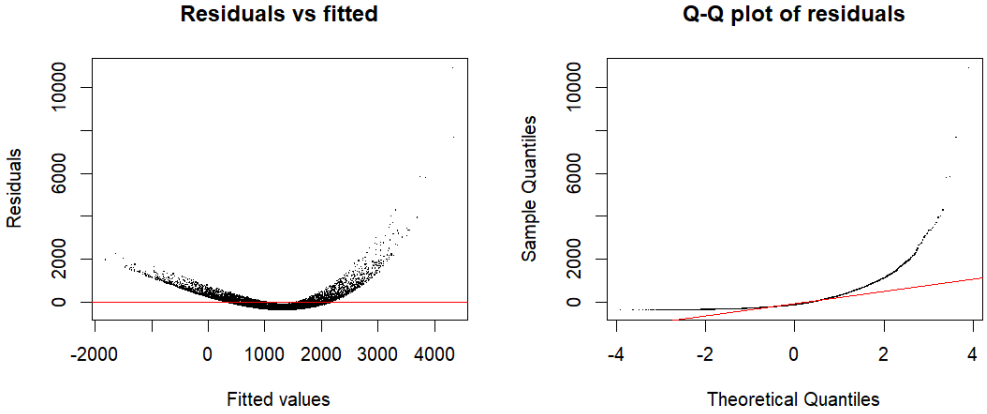


Figure 6: Residuals vs fitted values and normal Q-Q plot for the baseline linear model.

4 Task 2: Regression – Modelling the Relationship between Server Metrics

4.1 Task 2.1: Least squares parameter estimates

For each model a design matrix was built from the transformed inputs plus a column of ones, and Equation (1) was solved with `solve(t(X) %*% X) %*% t(X) %*% y` using all 10,000 samples (Appendix A.2, section "Task 2.1"). As a check, Model 1 was also fitted with `lm()`, which gave the same estimates to every printed digit. Table 3 lists the results.

Table 3: Least squares estimates $\hat{\boldsymbol{\theta}}$ for the five candidate models (all data).

Model	Term	Estimate
1	x_4 / x_1^2 / x_1^3	3.0940 / −4.0550 / 2.3840
	x_2^4 / x_1^4 / bias	−0.005608 / −0.09913 / 930.83
2	x_4 / x_1^3 / x_3^4	4.4805 / 1.20018 / 0.800003
	bias	120.022
3	x_3^3 / x_3^4 / bias	−0.03076 / 0.80153 / 308.585
4	x_2 / x_1^3 / x_3^4	0.1170 / 1.20032 / 0.79994
	bias	132.854
5	x_4 / x_1^2 / x_1^3	4.4805 / −0.01019 / 1.20146
	x_3^4 / bias	0.800003 / 120.083

The estimates already show a pattern. Every model that contains x_3^4 estimates its coefficient as about 0.80, and every model that contains x_1^3 estimates about 1.20. Models 2 and 5 give almost the same values for every shared term ($x_4 \approx 4.48$, bias ≈ 120), and the extra x_1^2 term in Model 5 is close to zero (−0.010). Model 1 has no x_3 term at all, so its intercept (930.8) is pulled towards the mean of y to make up for it.

4.2 Task 2.2: Residual sum of squares

The RSS of each model (Appendix A.2, "Task 2.2") is given in Table 4. The RSS values differ by more than four orders of magnitude. Models 2 and 5 have by far the lowest RSS (about 2.51×10^5). The models without the right terms are much worse: Model 4 swaps x_4 for x_2 (2.06×10^6), Model 3 has no CPU or network term (3.05×10^8), and Model 1 has no disk term (7.54×10^9). This agrees with Task 1, where x_3 was found to be the main driver of power.

4.3 Task 2.3: Log-likelihood

The variance $\hat{\sigma}^2 = \mathrm{RSS}/(n-1)$ and the log-likelihood from Equation (2) were computed for each model. For Models 2 and 5, $\hat{\sigma}^2 \approx 25.08$, so the noise standard deviation is about 5.0 W. This is very small next to the spread of y (884 W), and suggests that these models capture almost all

of the signal. Because $\hat{\sigma}^2$ is proportional to the RSS, the log-likelihood ranks the models in the same order as the RSS.

4.4 Task 2.4: AIC and BIC

AIC and BIC were computed from Equation (3) with k equal to the number of columns of each design matrix, including the bias. Table 4 collects all the criteria. ΔAIC and ΔBIC show the difference from the best model.

Table 4: Model comparison criteria (n = 10,000). The lowest AIC and BIC (selected model) are in bold.

Model	k	RSS	$\hat{\sigma}^2$	$\ln p(D\|\hat{\theta})$	AIC	BIC	ΔAIC	ΔBIC
1	6	7.538×10^9	753871.8	−81853.8	163719.6	163762.8	103111.7	103126.1
2	4	2.508×10^5	25.08	−30300.0	**60607.9**	**60636.7**	**0.0**	**0.0**
3	3	3.048×10^8	30478.8	−65812.8	131631.7	131653.3	71023.8	71016.5
4	4	2.056×10^6	205.65	−40819.7	81647.5	81676.3	21039.6	21039.6
5	5	2.508×10^5	25.08	−30299.8	60609.6	60645.6	1.7	8.9

Model 2 has the lowest AIC (60607.9) and the lowest BIC (60636.7). Model 5 comes a close second. Its RSS is only 8.3 lower than that of Model 2 (250802.2 against 250810.5), which raises its log-likelihood by just 0.17. That is far too little to pay for the extra parameter, so its AIC is 1.67 higher and its BIC 8.88 higher. Models 1, 3 and 4 are worse by tens of thousands of units and can be ruled out.

4.5 Task 2.5: Distribution of the residuals

Figure 7 shows a histogram and a normal Q-Q plot of the residuals of each model (Appendix A.2, "Task 2.5"). Since the noise in the data is Gaussian, the correct model should leave residuals that are Gaussian too.

- **Model 1**: the residuals are very right-skewed and range up to about 14,000. The Q-Q plot curves sharply upwards. The missing x_3^4 effect remains in the errors.
- **Model 3**: the residuals are bimodal and spread from −200 to 500, and the Q-Q plot has an S-shape. Leaving out x_1^3 creates this large structured error, because CPU utilisation is uniform and adds a flat-topped component.
- **Model 4**: the residuals are right-skewed with a long upper tail of up to about 100, because x_2 cannot stand in for the skewed network signal x_4.
- **Models 2 and 5**: the histograms are symmetric and bell-shaped, centred on zero and within about ±18. The Q-Q points lie on the reference line along the whole range. The two plots cannot be told apart.

The Shapiro–Wilk test, applied to the same random 5000 residuals for each model (the R function accepts at most 5000), agrees. Normality is not rejected for Model 2 ($p = 0.696$) or Model 5 ($p = 0.689$), but is strongly rejected for Models 1, 3 and 4 (all $p < 10^{-50}$).

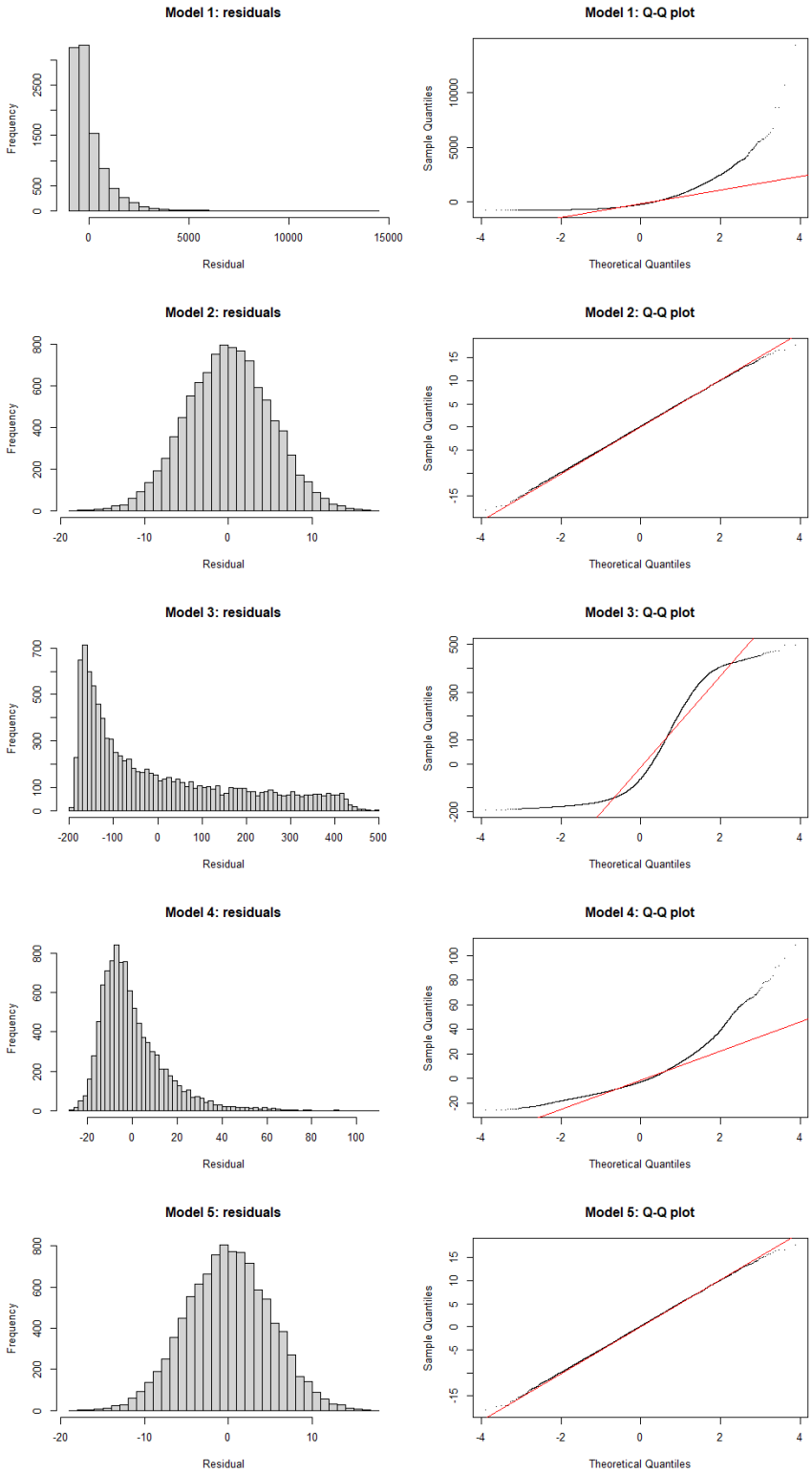


Figure 7: Residual histograms (left) and normal Q-Q plots (right) for the five candidate models.

4.6 Task 2.6: Selection of the best model

Model 2 was chosen as the best model:

$$\hat{y} = 4.4805\, x_4 + 1.2002\, x_1^3 + 0.8000\, x_3^4 + 120.02 \tag{4}$$

The reasons are:

1. **Information criteria:** it has the lowest AIC and the lowest BIC of the five models (`which.min()` returns 2 for both).
2. **Residuals:** its residuals are symmetric, close to normal on the Q-Q plot, and pass the Shapiro–Wilk test. This is what we expect when the model structure is right and only the Gaussian measurement noise is left.
3. **Parsimony:** Model 5 fits equally well but adds an x_1^2 term whose coefficient is almost zero (-0.010), so the term is not needed. According to Burnham and Anderson (2002), an AIC difference under 2 means that both models have support, so the decision rests on BIC, which favours Model 2 by 8.9 (strong evidence), and on choosing the simpler model. Model 2 is exactly Model 5 with $\theta_2 = 0$.

The estimated coefficients are also very close to round numbers (0.8, 1.2, 4.48 and 120). This suggests that the simulated data were produced by a model with this exact structure, so Model 2 is very likely the "true" model.

4.7 Task 2.7: Train/test validation and confidence intervals

The data were split at random into 7000 training and 3000 test rows (`set.seed(123)`, Appendix A.2, "Task 2.7"). The same row indices were used for $\mathbf{X}$ and $\mathbf{y}$. Model 2 was refitted on the training rows only.

1. Parameter estimates (training data). $\hat{\theta}_1 = 4.4640$ (x_4), $\hat{\theta}_2 = 1.20015$ (x_1^3), $\hat{\theta}_3 = 0.800002$ (x_3^4) and $\hat{\theta}_{bias} = 120.109$. These are almost identical to the full-data estimates, so the fit is stable.

2. Test predictions. On the 3000 unseen test rows the model reached an RMSE of 5.04 W and $R^2 = 0.99996$. The RMSE is almost exactly the noise level estimated in Task 2.3 ($\hat{\sigma} \approx 5.0$), so the model does not overfit: its test error is the noise floor. Figure 8 shows that all test predictions lie on the $y = x$ line.

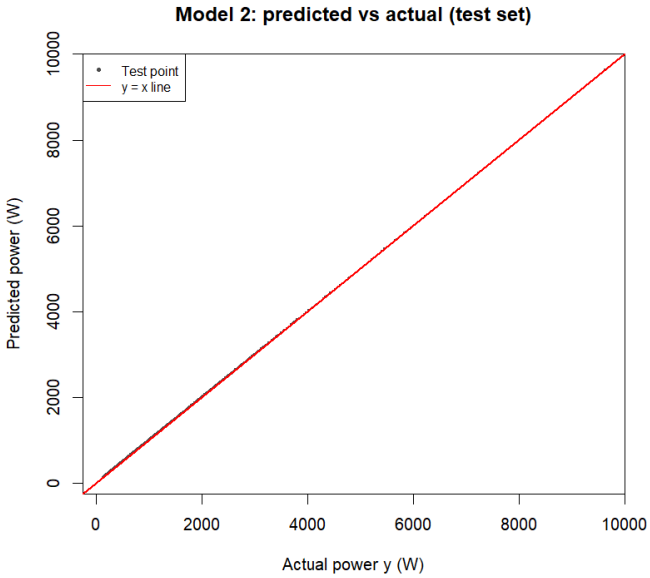


Figure 8: Predicted vs actual power for all 3000 test samples (Model 2 trained on 70% of the data).

3. 95% confidence intervals. The covariance of the estimates was computed as $\hat{\sigma}^2(\mathbf{X}_{tr}^T\mathbf{X}_{tr})^{-1}$ with $\hat{\sigma}^2 = \mathrm{RSS}_{tr}/(n_{tr} - k)$, and the critical value was $t_{0.975,6996} = 1.960$. Figure 9 shows the first 40 test samples. In the top panel the predictions (red crosses) lie on top of the observed values (grey dots). The confidence intervals are very narrow, with a median width of only 0.38 W (range 0.24–2.28 W). On a scale of thousands of watts the error bars are shorter than the plotting symbols, so the bottom panel plots each point relative to its prediction, where the error bars (blue) can be seen.

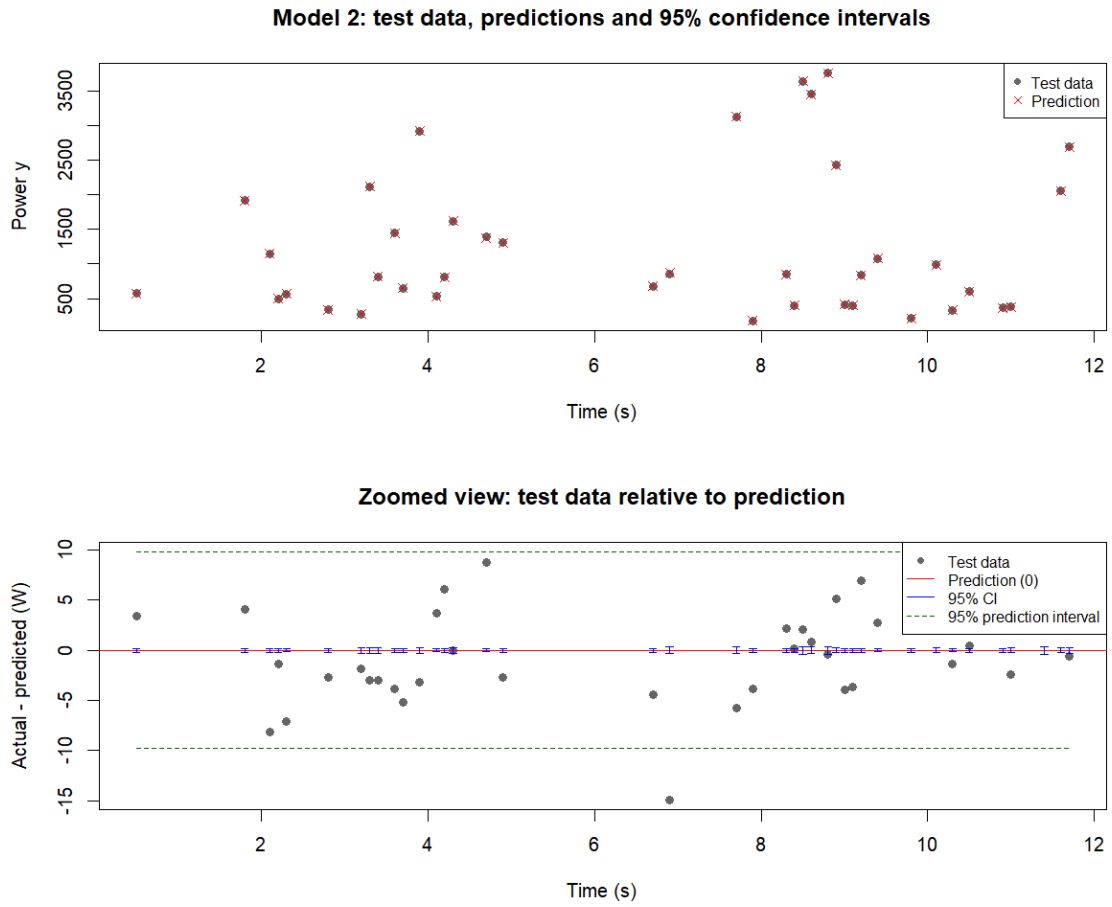


Figure 9: Model 2 predictions with 95% confidence intervals for the first 40 test samples. Top: actual vs predicted power. Bottom: zoomed view of actual minus predicted, with 95% confidence-interval error bars (blue) and 95% prediction interval (dashed green).

It is important to tell the two kinds of interval apart. The confidence interval shows the uncertainty in the *mean* prediction, which comes only from the uncertainty in $\hat{\boldsymbol{\theta}}$. With 7000 training samples this uncertainty is tiny, and only 3.7% of the actual test values fall inside these intervals. That is expected, not a failure. The prediction interval adds the noise of a single new observation and is about $\pm 9.8\,\mathrm{W}$ wide (total width $19.6\,\mathrm{W}$). It contains 94.6% of the test values, very close to the nominal 95%. This shows that the model and its error assumptions are well calibrated. The CI is widest for inputs far from the centre of the data, such as large x_3, because x_3^4 grows quickly there.

5 Task 3: Approximate Bayesian Computation (ABC)

Least squares gives one best value for each parameter. A Bayesian approach instead gives a full posterior distribution, which shows which values are plausible and how the parameters depend on each other. Rejection ABC approximates the posterior without writing down the likelihood: it only needs to simulate model output and compare it with the data (Beaumont et al., 2002; Sunnåker et al., 2013). The code is in Appendix A.3.

5.1 Task 3.1: Choice of parameters

The Model 2 estimates were ranked by absolute value with R's `order()` function (Appendix A.3, "Task 3.1"). The two largest are $\theta_{bias} = 120.022$ and $\theta_1 = 4.4805$ (the x_4 coefficient), so these two were estimated with ABC. The other two were fixed at their least squares values, $\theta_2 = 1.20018$ (x_1^3) and $\theta_3 = 0.800003$ (x_3^4).

5.2 Task 3.2: Uniform priors

A uniform prior was placed around each least squares estimate. To set a sensible width, the standard errors of the estimates were computed from $\hat{\sigma}^2(\mathbf{X}^T\mathbf{X})^{-1}$: $\mathrm{SE}(\theta_{bias}) = 0.0978$ and $\mathrm{SE}(\theta_1) = 0.0167$. Each prior was set to the estimate ± 10 standard errors:

$$\theta_{bias} \sim U(119.045,\ 121.000),$$
$$\theta_1 \sim U(4.3135,\ 4.6475).$$

This range is wide enough to cover all plausible values, since the true value lies within a few standard errors of the estimate. It is also narrow enough that a useful share of the draws is accepted. A prior of $\pm 50\%$ of the estimate would accept almost nothing, because these parameters are known so precisely.

5.3 Task 3.3: Rejection ABC

The algorithm used was:

1. Draw $N = 100{,}000$ pairs $(\theta_{bias}^{(j)}, \theta_1^{(j)})$ independently from the two uniform priors, with the random seed fixed at 456 so that the run can be repeated.
2. For each pair, compute the model output, with θ_2, θ_3 fixed:

$$\hat{\mathbf{y}}^{(j)} = \theta_{bias}^{(j)} + \theta_1^{(j)} x_4 + \theta_2 x_1^3 + \theta_3 x_3^4.$$

3. Measure the distance from the data as $\mathrm{RSS}^{(j)} = \sum_i (y_i - \hat{y}_i^{(j)})^2$.
4. Accept the pair if $\mathrm{RSS}^{(j)} \le \epsilon$, with $\epsilon = 1.0005 \times \mathrm{RSS}_{LS} = 250935.9$, that is, if it fits within 0.05% of the best possible fit.

The tolerance allows the RSS to rise by 125.4, which is about $5\hat{\sigma}^2$, or a drop of about 2.5 in the log-likelihood. A tolerance on this scale keeps parameter values that are statistically compatible with the data. The RSS of the prior draws ranged from 250,811 to 275,113. In total, **1921 draws were accepted** (an acceptance rate of 1.92%), enough for stable histograms.

5.4 Task 3.4: Joint and marginal posterior distributions

Figure 10 shows the accepted samples. The left panel is the joint posterior and the other two panels are the marginal posteriors. The red markers show the least squares estimates. Table 5 summarises the posteriors.

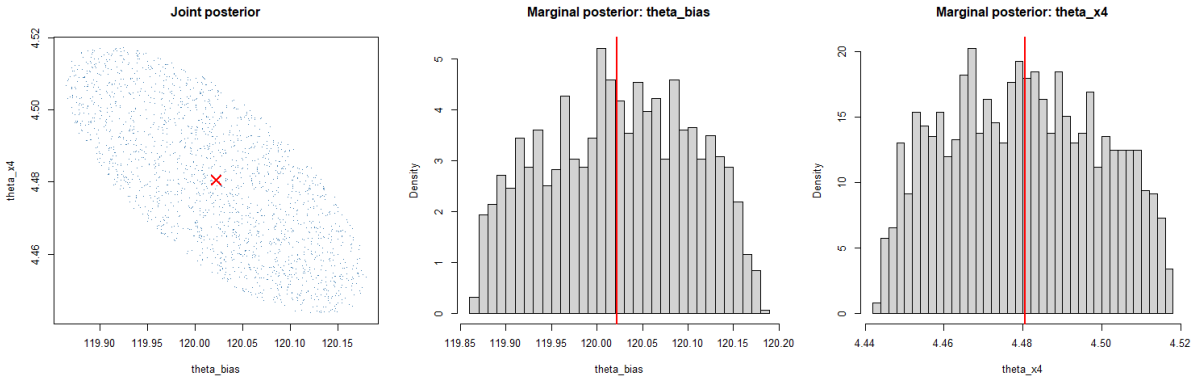


Figure 10: Rejection ABC results for Model 2: joint posterior of $(\theta_{bias}, \theta_{x_4})$ (left) and marginal posteriors (centre, right). The red markers show the least squares estimates.

Table 5: Posterior summaries from 1921 accepted ABC samples.

Parameter	LS est.	Post. mean	Post. SD	95% cred. interval	Prior range
θ_{bias}	120.022	120.023	0.079	[119.881, 120.157]	[119.045, 121.000]
θ_1 (x_4)	4.4805	4.4799	0.019	[4.4479, 4.5134]	[4.3135, 4.6475]

5.5 Task 3.5: Explanation of the results

Agreement with least squares. Both posterior means agree with the least squares estimates to three or four significant figures, and both estimates lie in the centre of the joint cloud. This is expected: with a flat prior, the posterior is driven by the likelihood, and least squares is the maximum likelihood estimate.

Learning from the data. The posterior ranges are much narrower than the priors. The 95% credible interval of θ_{bias} (width 0.28) covers only about 14% of the prior width of 1.96, and that of θ_1 (width 0.066) about 20% of the prior width of 0.33. The data have greatly reduced the uncertainty. Both intervals exclude zero by a wide margin, so network traffic has a clear positive effect on power ($\approx 4.5\,\mathrm{W}$ per unit of x_4), even though Task 1 showed almost no correlation between x_4 and y on its own. The idle power is about 120 W.

Correlation between parameters. The joint posterior is a tilted ellipse with correlation -0.71. Network traffic is always positive (mean 3.0), so a larger θ_1 raises every prediction by about $3\,\Delta\theta_1$. A smaller intercept can cancel this. The data cannot fully separate the two effects, so they trade off against each other. This information is lost if the parameters are reported one at a time.

Shape of the posterior. The marginal histograms are rounded and fairly flat on top rather

than bell-shaped. This comes from the method. A fixed RSS threshold accepts every draw inside an ellipse of nearly equal fit, and because the prior is uniform, the accepted points are spread almost evenly inside that ellipse. A smaller tolerance (with more draws), or a version that adds simulated Gaussian noise to each output, would give a shape closer to the true Gaussian posterior. The location, spread and correlation would stay much the same.

6 Discussion

What the model says about the server. The selected model (Equation 4) splits power into four parts:

- a fixed idle power of about 120 W, drawn even when the load is zero, which matches the observation that servers are not energy proportional (Barroso and Hölzle, 2007);
- a cubic CPU term ($1.2\,x_1^3$), which fits the common view that dynamic CPU power rises faster than linearly as clock speed and voltage increase together;
- a fourth-power disk I/O term ($0.8\,x_3^4$), which is by far the largest source of variation and causes the rare, very high power peaks;
- a small linear network term ($4.48\,x_4$).

Memory usage (x_2) does not appear in the selected model. Model 4, which uses x_2 in place of x_4, was much worse ($\Delta\text{BIC} > 21{,}000$), and x_2 was not significant in the baseline linear model. Because the data are simulated, these terms describe how the data were generated and should not be read as a physical law. The pattern is still realistic: storage-heavy and compute-heavy workloads drive power far more than memory use does.

Agreement between methods. Several independent pieces of evidence point to Model 2: the large drop in RSS, the lowest AIC and BIC, Gaussian residuals, a test RMSE equal to the noise level, a 95% prediction interval with 94.6% coverage, and ABC posteriors centred on the least squares values. Each check tests a different property, such as in-sample fit, model complexity, error structure, out-of-sample accuracy, interval calibration and parameter uncertainty. Their agreement gives strong confidence in the choice.

Exploratory analysis can mislead. Task 1 alone would have suggested that network traffic does not matter, since $r = 0.010$. Yet the regression shows a clear effect with a very small standard error (0.017), and leaving out x_4 (as Model 4 does) raises the RSS by a factor of eight. A small effect can be hidden in pairwise plots by the much larger variation from another input. Variables should therefore not be dropped on the basis of correlations alone.

Choosing between nested models. Model 5 contains Model 2, so its RSS is always at least as low. Choosing a model by RSS or R^2 alone would pick the larger model every time. AIC and BIC penalise the extra parameter. In this case $\Delta\text{AIC} = 1.67$ is borderline, while $\Delta\text{BIC} = 8.9$ is decisive. With large samples BIC is consistent: it tends to pick the true model when that

model is among the candidates (Schwarz, 1978; Burnham and Anderson, 2002). This matches what we see here.

Limitations.

- The brief states that the *inputs* also contain noise. Least squares assumes the inputs are measured exactly. Noise in the inputs biases the estimates towards zero, and the bias grows when inputs are raised to high powers. The coefficients here are so close to round numbers that the input noise seems to be small, but in real telemetry this could matter.
- The variance in Task 2.3 uses $n - 1$, as the brief requires, rather than the unbiased $n - k$. With $n = 10{,}000$ the difference is negligible and does not change the ranking.
- The Shapiro–Wilk test was run on a random sample of 5000 residuals. With very large samples, formal tests can reject normality for tiny departures, so the Q-Q plots are the main evidence.
- The ABC distance is computed from the deterministic model output, without simulated noise, and with a fixed RSS tolerance. As explained in Section 5, this shapes the posterior. Only two of the four parameters were treated as uncertain, so the posterior is conditional on the other two being fixed, and it understates the full uncertainty.
- Only the five given models were compared. A model outside this set, for example one with interaction terms, could not be found by this procedure.

Reflection. Writing the least squares solution by hand instead of only calling `lm()` made each step clear, and checking against `lm()` gave confidence that the code was correct. The most useful lesson was to look at residual plots and test-set behaviour as well as the numbers. For example, the 3.7% coverage of the confidence interval looked like an error at first, but it is the correct result once the confidence interval is separated from the prediction interval.

7 Conclusion

This report modelled simulated cloud server power consumption from four telemetry inputs using polynomial regression. The exploratory analysis showed stationary, independent samples. Disk I/O was strongly and non-linearly linked to power, CPU weakly, and memory and network showed no visible link. A linear model explained only 76% of the variance and left clearly non-Gaussian residuals.

Of the five candidate models, **Model 2**, $y = 4.48\,x_4 + 1.20\,x_1^3 + 0.80\,x_3^4 + 120.0$, was selected. It had the lowest AIC (60607.9) and BIC (60636.7), and its residuals were the only ones, together with those of the over-parameterised Model 5, that were consistent with Gaussian noise of standard deviation about 5 W. When trained on 70% of the data, it predicted the remaining 30% with an RMSE of 5.04 W and $R^2 = 0.99996$, and its 95% prediction intervals covered 94.6% of the test values.

Rejection ABC on the two largest parameters gave posterior means of 120.02 for the intercept and 4.480 for the network coefficient, with narrow 95% credible intervals and a negative correlation of -0.71 between them. These results confirm the least squares estimates and add information about how the two parameters depend on each other.

In practice, a model of this kind could estimate server power from software telemetry alone, without hardware meters, and help with energy-aware scheduling. Future work could use errors-in-variables regression to handle noise in the inputs, ABC on all four parameters with a smaller tolerance, and testing on real server measurements.

References

Akaike, H. (1974) ‘A new look at the statistical model identification’, *IEEE Transactions on Automatic Control*, 19(6), pp. 716–723.

Barroso, L.A. and Hölzle, U. (2007) ‘The case for energy-proportional computing’, *Computer*, 40(12), pp. 33–37.

Beaumont, M.A., Zhang, W. and Balding, D.J. (2002) ‘Approximate Bayesian computation in population genetics’, *Genetics*, 162(4), pp. 2025–2035.

Burnham, K.P. and Anderson, D.R. (2002) *Model Selection and Multimodel Inference: A Practical Information-Theoretic Approach*. 2nd edn. New York: Springer.

Fan, X., Weber, W.-D. and Barroso, L.A. (2007) ‘Power provisioning for a warehouse-sized computer’, in *Proceedings of the 34th Annual International Symposium on Computer Architecture (ISCA ’07)*, pp. 13–23.

James, G., Witten, D., Hastie, T. and Tibshirani, R. (2013) *An Introduction to Statistical Learning: with Applications in R*. New York: Springer.

Montgomery, D.C., Peck, E.A. and Vining, G.G. (2012) *Introduction to Linear Regression Analysis*. 5th edn. Hoboken, NJ: Wiley.

Pritchard, J.K., Seielstad, M.T., Perez-Lezaun, A. and Feldman, M.W. (1999) ‘Population growth of human Y chromosomes: a study of Y chromosome microsatellites’, *Molecular Biology and Evolution*, 16(12), pp. 1791–1798.

R Core Team (2026) *R: A Language and Environment for Statistical Computing*. Version 4.6.1. Vienna, Austria: R Foundation for Statistical Computing.

Schwarz, G. (1978) ‘Estimating the dimension of a model’, *The Annals of Statistics*, 6(2), pp. 461–464.

Shapiro, S.S. and Wilk, M.B. (1965) ‘An analysis of variance test for normality (complete samples)’, *Biometrika*, 52(3–4), pp. 591–611.

Sunnåker, M., Busetto, A.G., Numminen, E., Corander, J., Foll, M. and Dessimoz, C. (2013) ‘Approximate Bayesian computation’, *PLoS Computational Biology*, 9(1), e1002803.

A R Code

All code is written in base R (version 4.6.1) and is split into three scripts, one for each task. Each script is run from the project folder that contains `data/`, `scripts/` and `figures/`. The Task 3 script reuses the objects created in Task 2 and runs the Task 2 script itself if they do not exist yet. Comments in the code mark each sub-task (Task 1.1, Task 2.3, and so on), and these match the section headings in the report.

A.1 Task 1: Preliminary data analysis

Listing 1: Task 1 – `task1_data_exploration.R`

```r
# Task 1: Preliminary data analysis
# Run from the project folder (the one that contains data/, scripts/, figures/)

# load the data files
X    <- read.csv("data/X.csv", header = TRUE)
y    <- read.csv("data/y.csv", header = TRUE)
time <- read.csv("data/time.csv", header = TRUE)

# put everything in one table
df <- data.frame(time = time$time_seconds,
                 x1 = X$cpu_utilization,
                 x2 = X$memory_usage,
                 x3 = X$disk_io,
                 x4 = X$network_traffic,
                 y  = y$power_consumption)

# check for missing values and look at summary
colSums(is.na(df))
summary(df)

# standard deviation and skewness of each signal
skew <- function(v) mean((v - mean(v))^3) / sd(v)^3
round(sapply(df[, 2:6], sd), 3)
round(sapply(df[, 2:6], skew), 3)

dir.create("figures", showWarnings = FALSE)

# ---- Task 1.1: Time series plots ----

# full 1000 s record (10000 samples)
png("figures/fig1_time_series.png", width = 1000, height = 1400, res = 120)
par(mfrow = c(5, 1))

plot(df$time, df$x1, type = "l", main = "x1: CPU utilization", xlab = "Time (s)", ylab = "x1")
plot(df$time, df$x2, type = "l", main = "x2: Memory usage", xlab = "Time (s)", ylab = "x2")
plot(df$time, df$x3, type = "l", main = "x3: Disk I/O", xlab = "Time (s)", ylab = "x3")
plot(df$time, df$x4, type = "l", main = "x4: Network traffic", xlab = "Time (s)", ylab = "x4")
plot(df$time, df$y, type = "l", main = "y: Power consumption", xlab = "Time (s)", ylab = "y")

```

```r
dev.off()

# zoomed view of the first 20 seconds (200 samples) - the full plot is too dense to read
zoom <- df$time <= 20

png("figures/fig1b_time_series_zoom.png", width = 1000, height = 1400, res = 120)
par(mfrow = c(5, 1))

plot(df$time[zoom], df$x1[zoom], type = "o", pch = 20, cex = 0.5,
     main = "x1: CPU utilization (first 20 s)", xlab = "Time (s)", ylab = "x1")
plot(df$time[zoom], df$x2[zoom], type = "o", pch = 20, cex = 0.5,
     main = "x2: Memory usage (first 20 s)", xlab = "Time (s)", ylab = "x2")
plot(df$time[zoom], df$x3[zoom], type = "o", pch = 20, cex = 0.5,
     main = "x3: Disk I/O (first 20 s)", xlab = "Time (s)", ylab = "x3")
plot(df$time[zoom], df$x4[zoom], type = "o", pch = 20, cex = 0.5,
     main = "x4: Network traffic (first 20 s)", xlab = "Time (s)", ylab = "x4")
plot(df$time[zoom], df$y[zoom], type = "o", pch = 20, cex = 0.5,
     main = "y: Power consumption (first 20 s)", xlab = "Time (s)", ylab = "y")

dev.off()

# autocorrelation at lag 1 - checks whether samples depend on the previous sample
round(sapply(df[, 2:6], function(v) acf(v, lag.max = 1, plot = FALSE)$acf[2]), 3)

# ---- Task 1.2: Distribution of each signal ----

png("figures/fig2_distributions.png", width = 1200, height = 800, res = 120)
par(mfrow = c(2, 3))

hist(df$x1, breaks = 30, freq = FALSE, main = "x1: CPU utilization", xlab = "x1")
lines(density(df$x1), col = "red")

hist(df$x2, breaks = 30, freq = FALSE, main = "x2: Memory usage", xlab = "x2")
lines(density(df$x2), col = "red")

hist(df$x3, breaks = 30, freq = FALSE, main = "x3: Disk I/O", xlab = "x3")
lines(density(df$x3), col = "red")

hist(df$x4, breaks = 30, freq = FALSE, main = "x4: Network traffic", xlab = "x4")
lines(density(df$x4), col = "red")

# y has a long right tail, so the y-axis limit is set from the density curve
d_y <- density(df$y)
hist(df$y, breaks = 60, freq = FALSE, ylim = c(0, max(d_y$y) * 1.1),
     main = "y: Power consumption", xlab = "y")
lines(d_y, col = "red")

dev.off()

# ---- Task 1.3: Correlation and scatter plots ----

# correlation between all signals (time column left out)
```

```r
cor_pearson <- cor(df[, 2:6], method = "pearson")
round(cor_pearson, 3)

cor_spearman <- cor(df[, 2:6], method = "spearman")
round(cor_spearman, 3)

# correlation heatmap (Pearson) with the values written in each cell
png("figures/fig3a_correlation.png", width = 800, height = 700, res = 120)
par(mar = c(4, 4, 3, 6))
vars <- colnames(cor_pearson)
cols <- colorRampPalette(c("#2166ac", "white", "#b2182b"))(41)
image(1:5, 1:5, cor_pearson[, 5:1], zlim = c(-1, 1), col = cols,
      axes = FALSE, xlab = "", ylab = "", main = "Pearson correlation matrix")
axis(1, at = 1:5, labels = vars)
axis(2, at = 1:5, labels = rev(vars), las = 1)
for (i in 1:5) for (j in 1:5)
  text(i, 6 - j, sprintf("%.2f", cor_pearson[i, j]), cex = 0.9)
box()
dev.off()

# scatter plots of each input against power
png("figures/fig3_scatter.png", width = 1100, height = 900, res = 120)
par(mfrow = c(2, 2))

plot(df$x1, df$y, pch = ".", main = "y vs x1 (CPU)", xlab = "x1", ylab = "y")
lines(lowess(df$x1, df$y), col = "red", lwd = 2)

plot(df$x2, df$y, pch = ".", main = "y vs x2 (Memory)", xlab = "x2", ylab = "y")
lines(lowess(df$x2, df$y), col = "red", lwd = 2)

plot(df$x3, df$y, pch = ".", main = "y vs x3 (Disk I/O)", xlab = "x3", ylab = "y")
lines(lowess(df$x3, df$y), col = "red", lwd = 2)

plot(df$x4, df$y, pch = ".", main = "y vs x4 (Network)", xlab = "x4", ylab = "y")
lines(lowess(df$x4, df$y), col = "red", lwd = 2)

dev.off()

# baseline linear model with all four inputs
model_lin <- lm(y ~ x1 + x2 + x3 + x4, data = df)
summary(model_lin)

# residual plots for the linear model
png("figures/fig4_linear_residuals.png", width = 1100, height = 500, res = 120)
par(mfrow = c(1, 2))

plot(fitted(model_lin), resid(model_lin), pch = ".",
     main = "Residuals vs fitted", xlab = "Fitted values", ylab = "Residuals")
abline(h = 0, col = "red")

qqnorm(resid(model_lin), pch = ".", main = "Q-Q plot of residuals")
qqline(resid(model_lin), col = "red")
```

```
144
145 dev.off()
```

A.2 Task 2: Regression (Tasks 2.1–2.7)

Listing 2: Task 2 – `task2_regression.R`

```r
# Task 2: Polynomial regression models
# load the data files
X    <- read.csv("data/X.csv", header = TRUE)
y    <- read.csv("data/y.csv", header = TRUE)
time <- read.csv("data/time.csv", header = TRUE)

df <- data.frame(time = time$time_seconds,
                 x1 = X$cpu_utilization,
                 x2 = X$memory_usage,
                 x3 = X$disk_io,
                 x4 = X$network_traffic,
                 y  = y$power_consumption)

head(df)

# ---- Task 2.1: Least squares estimation ----

# output as a plain vector
y_vec <- df$y

# column of ones for the bias term
ones <- rep(1, nrow(df))

# Model 1: x4, x1^2, x1^3, x2^4, x1^4, bias
X_m1 <- cbind(x4 = df$x4, x1_2 = df$x1^2, x1_3 = df$x1^3,
              x2_4 = df$x2^4, x1_4 = df$x1^4, bias = ones)

theta_m1 <- solve(t(X_m1) %*% X_m1) %*% t(X_m1) %*% y_vec
theta_m1

# check Model 1 against lm() - estimates should match theta_m1
check_m1 <- lm(y ~ x4 + I(x1^2) + I(x1^3) + I(x2^4) + I(x1^4), data = df)
coef(check_m1)

# Model 2: x4, x1^3, x3^4, bias
X_m2 <- cbind(x4 = df$x4, x1_3 = df$x1^3, x3_4 = df$x3^4, bias = ones)
theta_m2 <- solve(t(X_m2) %*% X_m2) %*% t(X_m2) %*% y_vec
theta_m2

# Model 3: x3^3, x3^4, bias
X_m3 <- cbind(x3_3 = df$x3^3, x3_4 = df$x3^4, bias = ones)
theta_m3 <- solve(t(X_m3) %*% X_m3) %*% t(X_m3) %*% y_vec
theta_m3

```

```r
# Model 4: x2, x1^3, x3^4, bias
X_m4 <- cbind(x2 = df$x2, x1_3 = df$x1^3, x3_4 = df$x3^4, bias = ones)
theta_m4 <- solve(t(X_m4) %*% X_m4) %*% t(X_m4) %*% y_vec
theta_m4

# Model 5: x4, x1^2, x1^3, x3^4, bias
X_m5 <- cbind(x4 = df$x4, x1_2 = df$x1^2, x1_3 = df$x1^3,
              x3_4 = df$x3^4, bias = ones)
theta_m5 <- solve(t(X_m5) %*% X_m5) %*% t(X_m5) %*% y_vec
theta_m5

# ---- Task 2.2: Residual sum of squares (RSS) ----

# predictions from each model
yhat_m1 <- X_m1 %*% theta_m1
yhat_m2 <- X_m2 %*% theta_m2
yhat_m3 <- X_m3 %*% theta_m3
yhat_m4 <- X_m4 %*% theta_m4
yhat_m5 <- X_m5 %*% theta_m5

# RSS = sum of squared errors
rss_m1 <- sum((y_vec - yhat_m1)^2)
rss_m2 <- sum((y_vec - yhat_m2)^2)
rss_m3 <- sum((y_vec - yhat_m3)^2)
rss_m4 <- sum((y_vec - yhat_m4)^2)
rss_m5 <- sum((y_vec - yhat_m5)^2)

rss_all <- c(rss_m1, rss_m2, rss_m3, rss_m4, rss_m5)
rss_all

# ---- Task 2.3: Log-likelihood ----

n <- length(y_vec)            # number of samples (10000)

# estimated noise variance for each model
sigma2_all <- rss_all / (n - 1)

# log-likelihood for each model
loglik_all <- -(n / 2) * log(2 * pi) - (n / 2) * log(sigma2_all) - rss_all / (2 * sigma2_all)

sigma2_all
loglik_all

# ---- Task 2.4: AIC and BIC ----

# number of parameters in each model (including bias)
k_all <- c(ncol(X_m1), ncol(X_m2), ncol(X_m3), ncol(X_m4), ncol(X_m5))
k_all

aic_all <- 2 * k_all - 2 * loglik_all
bic_all <- k_all * log(n) - 2 * loglik_all

```

```r
# summary table of all results so far
results <- data.frame(model  = 1:5,
                      k      = k_all,
                      RSS    = rss_all,
                      logLik = loglik_all,
                      AIC    = aic_all,
                      BIC    = bic_all)
results

# ---- Task 2.5: Residual distributions ----

# residuals (errors) for each model
res_m1 <- as.vector(y_vec - yhat_m1)
res_m2 <- as.vector(y_vec - yhat_m2)
res_m3 <- as.vector(y_vec - yhat_m3)
res_m4 <- as.vector(y_vec - yhat_m4)
res_m5 <- as.vector(y_vec - yhat_m5)

png("figures/fig5_residuals.png", width = 1000, height = 1800, res = 120)
par(mfrow = c(5, 2))

hist(res_m1, breaks = 50, main = "Model 1: residuals", xlab = "Residual")
qqnorm(res_m1, pch = ".", main = "Model 1: Q-Q plot"); qqline(res_m1, col = "red")

hist(res_m2, breaks = 50, main = "Model 2: residuals", xlab = "Residual")
qqnorm(res_m2, pch = ".", main = "Model 2: Q-Q plot"); qqline(res_m2, col = "red")

hist(res_m3, breaks = 50, main = "Model 3: residuals", xlab = "Residual")
qqnorm(res_m3, pch = ".", main = "Model 3: Q-Q plot"); qqline(res_m3, col = "red")

hist(res_m4, breaks = 50, main = "Model 4: residuals", xlab = "Residual")
qqnorm(res_m4, pch = ".", main = "Model 4: Q-Q plot"); qqline(res_m4, col = "red")

hist(res_m5, breaks = 50, main = "Model 5: residuals", xlab = "Residual")
qqnorm(res_m5, pch = ".", main = "Model 5: Q-Q plot"); qqline(res_m5, col = "red")

dev.off()

# Shapiro-Wilk normality test on a random 5000 residuals (the test allows at most 5000)
set.seed(1)
sw_idx <- sample(1:n, 5000)
sapply(list(m1 = res_m1, m2 = res_m2, m3 = res_m3, m4 = res_m4, m5 = res_m5),
       function(r) shapiro.test(r[sw_idx])$p.value)

# ---- Task 2.6: Model selection ----

# lowest AIC and lowest BIC
which.min(aic_all)
which.min(bic_all)

# difference from the best model (smaller = better, 0 = best)
results$dAIC <- aic_all - min(aic_all)
```

```r
results$dBIC <- bic_all - min(bic_all)
results

# ---- Task 2.7: Train/test split and prediction ----

set.seed(123)   # makes the random split the same every time the script runs

# randomly choose 70% of the row numbers for training
train_idx <- sample(1:n, size = round(0.7 * n))

# Model 2 design matrix, split by the SAME rows for X and y
X_train <- X_m2[train_idx, ]
y_train <- y_vec[train_idx]
X_test  <- X_m2[-train_idx, ]
y_test  <- y_vec[-train_idx]

nrow(X_train)   # should be 7000
nrow(X_test)    # should be 3000

# 1. estimate parameters using the training data only
theta_train <- solve(t(X_train) %*% X_train) %*% t(X_train) %*% y_train
theta_train

# 2. predict the test data
yhat_test <- as.vector(X_test %*% theta_train)

# how good are the test predictions?
rmse_test <- sqrt(mean((y_test - yhat_test)^2))
r2_test   <- 1 - sum((y_test - yhat_test)^2) / sum((y_test - mean(y_test))^2)
rmse_test
r2_test

# 3. 95% confidence intervals for the test predictions

k <- ncol(X_train)                      # number of parameters (4)
rss_train    <- sum((y_train - X_train %*% theta_train)^2)
sigma2_train <- rss_train / (nrow(X_train) - k)   # noise variance

cov_theta <- sigma2_train * solve(t(X_train) %*% X_train)   # parameter uncertainty

# variance of each test prediction: x*' Cov(theta) x*, done for all rows at once
var_fit <- rowSums((X_test %*% cov_theta) * X_test)
se_fit  <- sqrt(var_fit)

t_crit <- qt(0.975, df = nrow(X_train) - k)   # t-value for 95%

ci_lower <- yhat_test - t_crit * se_fit
ci_upper <- yhat_test + t_crit * se_fit

# prediction interval: also includes the noise in a single new observation
pi_lower <- yhat_test - t_crit * sqrt(var_fit + sigma2_train)
pi_upper <- yhat_test + t_crit * sqrt(var_fit + sigma2_train)
```

```r

t_crit
summary(ci_upper - ci_lower)   # width of the confidence intervals
summary(pi_upper - pi_lower)   # width of the prediction intervals

# what share of actual test values fall inside each interval?
mean(y_test >= ci_lower & y_test <= ci_upper)
mean(y_test >= pi_lower & y_test <= pi_upper)

# 4. plot predictions, 95% CIs and test data (first 40 test points)
idx    <- 1:40
t_test <- df$time[-train_idx][idx]     # time of each test point

png("figures/fig6_confidence_intervals.png", width = 1200, height = 1000, res = 120)
par(mfrow = c(2, 1))

# panel 1: actual values, predictions and CI error bars
plot(t_test, y_test[idx], pch = 16, col = "grey40",
     main = "Model 2: test data, predictions and 95% confidence intervals",
     xlab = "Time (s)", ylab = "Power y")
points(t_test, yhat_test[idx], pch = 4, col = "red")
# the CI error bars are under 1 W wide, too short to draw on this scale,
# so they are shown in the zoomed panel below
legend("topright", legend = c("Test data", "Prediction"),
       pch = c(16, 4), col = c("grey40", "red"), cex = 0.8)

# panel 2: zoomed view - distance of each point from its prediction
dev_y  <- y_test[idx] - yhat_test[idx]
plot(t_test, dev_y, pch = 16, col = "grey40",
     ylim = range(c(dev_y, pi_lower[idx] - yhat_test[idx], pi_upper[idx] - yhat_test[idx])),
     main = "Zoomed view: test data relative to prediction",
     xlab = "Time (s)", ylab = "Actual - predicted (W)")
abline(h = 0, col = "red")
arrows(t_test, ci_lower[idx] - yhat_test[idx], t_test, ci_upper[idx] - yhat_test[idx],
       angle = 90, code = 3, length = 0.03, col = "blue")
lines(t_test, pi_lower[idx] - yhat_test[idx], lty = 2, col = "darkgreen")
lines(t_test, pi_upper[idx] - yhat_test[idx], lty = 2, col = "darkgreen")
legend("topright", legend = c("Test data", "Prediction (0)", "95% CI", "95% prediction interval"
    ),
       pch = c(16, NA, NA, NA), lty = c(NA, 1, 1, 2),
       col = c("grey40", "red", "blue", "darkgreen"), cex = 0.8)

dev.off()

# predicted vs actual for all 3000 test points
png("figures/fig6b_pred_vs_actual.png", width = 800, height = 750, res = 120)
plot(y_test, yhat_test, pch = 20, cex = 0.4, col = "grey30",
     main = "Model 2: predicted vs actual (test set)",
     xlab = "Actual power y (W)", ylab = "Predicted power (W)")
abline(0, 1, col = "red", lwd = 2)
legend("topleft", legend = c("Test point", "y = x line"),
       pch = c(20, NA), lty = c(NA, 1), col = c("grey30", "red"), cex = 0.8)
```

```r
dev.off()
```

A.3 Task 3: Approximate Bayesian Computation

Listing 3: Task 3 – `task3_abc.R`

```r
# Task 3: Approximate Bayesian Computation (rejection ABC) for Model 2
# Needs the objects from Task 2 (df, y_vec, X_m2, theta_m2), so run Task 2 first if they are missing
if (!exists("theta_m2")) source("scripts/task2_regression.R")

# ---- Task 3.1: Choose the 2 parameters with the largest |theta| ----
theta_m2
order(abs(theta_m2), decreasing = TRUE)

# ---- Task 3.2: Uniform priors around the estimates ----
# standard errors of the least squares estimates, used to set the prior width
n <- length(y_vec)
k <- ncol(X_m2)

rss_m2    <- sum((y_vec - X_m2 %*% theta_m2)^2)
sigma2_m2 <- rss_m2 / (n - k)

se_m2 <- sqrt(diag(sigma2_m2 * solve(t(X_m2) %*% X_m2)))
se_m2

# ---- Task 3.3: Sample from the priors and run rejection ABC ----

# fixed parameters (x1^3 and x3^4 terms stay at their least squares values)
fixed_part <- theta_m2[2] * df$x1^3 + theta_m2[3] * df$x3^4

# uniform priors: estimate +/- 10 standard errors
prior_bias <- c(theta_m2[4] - 10 * se_m2[4], theta_m2[4] + 10 * se_m2[4])
prior_x4   <- c(theta_m2[1] - 10 * se_m2[1], theta_m2[1] + 10 * se_m2[1])
prior_bias
prior_x4

# draw samples from the priors
set.seed(456)
N <- 100000
bias_samples <- runif(N, prior_bias[1], prior_bias[2])
x4_samples   <- runif(N, prior_x4[1],   prior_x4[2])

# tolerance: accept fits within 0.05% of the least squares RSS
eps <- rss_m2 * 1.0005

# compute the RSS for every sample
rss_samples <- numeric(N)
for (i in 1:N) {
  y_sim <- bias_samples[i] + x4_samples[i] * df$x4 + fixed_part
  rss_samples[i] <- sum((y_vec - y_sim)^2)
```

```r
}

# keep the samples that fit well enough
accept    <- rss_samples <= eps
post_bias <- bias_samples[accept]
post_x4   <- x4_samples[accept]

sum(accept)     # number accepted
mean(accept)    # acceptance rate

# ---- Task 3.5: Posterior summaries (used to explain the results) ----

# posterior mean and 95% credible interval for each parameter
mean(post_bias)
quantile(post_bias, c(0.025, 0.975))

mean(post_x4)
quantile(post_x4, c(0.025, 0.975))

# correlation between the two parameters in the posterior
cor(post_bias, post_x4)

# ---- Task 3.4: Plot joint and marginal posteriors ----
png("figures/fig7_abc_posterior.png", width = 1500, height = 500, res = 120)
par(mfrow = c(1, 3))

# joint posterior
plot(post_bias, post_x4, pch = ".", col = "steelblue",
     main = "Joint posterior", xlab = "theta_bias", ylab = "theta_x4")
points(theta_m2[4], theta_m2[1], pch = 4, col = "red", cex = 2, lwd = 2)

# marginal posterior of theta_bias
hist(post_bias, breaks = 40, freq = FALSE, col = "lightgrey",
     main = "Marginal posterior: theta_bias", xlab = "theta_bias")
abline(v = theta_m2[4], col = "red", lwd = 2)

# marginal posterior of theta_x4
hist(post_x4, breaks = 40, freq = FALSE, col = "lightgrey",
     main = "Marginal posterior: theta_x4", xlab = "theta_x4")
abline(v = theta_m2[1], col = "red", lwd = 2)

dev.off()
```